Why Canonical-Round Algorithms Fail for Optimal Byzantine Resilience

Hagit Attiya
Technion – Israel Institute of Technology
Haifa, Israel
hagit@cs.technion.ac.il

Itay Flam
Technion – Israel Institute of Technology
Haifa, Israel
itay.flam@campus.technion.ac.il

Jennifer L. Welch
Texas A&M University
College Station, TX, USA
welch@cse.tamu.edu

Abstract

Canonical asynchronous rounds are a widely used abstraction for structuring distributed algorithms, making asynchronous executions appear synchronous and enabling modular reasoning. We show that this abstraction is fundamentally incompatible with optimal resilience in the Byzantine setting, even when randomization is allowed. Specifically, we prove that when $3f < n \leq 5f$, where n is the number of processes and at most f may be Byzantine faulty, no randomized canonical-round algorithm can solve consensus with bounded expected round complexity, and that communication-closed variants fail to solve consensus altogether in this regime.

We establish these lower bounds via a unifying notion of *non-trivial convergence*, which captures consensus as well as classical relaxations, such as approximate agreement and connected consensus. Using simple reductions, the same impossibility extends to fundamental communication primitives such as reliable broadcast and gather.

Our results identify a sharp boundary: while bounded canonical-round algorithms for these problems exist when $n > 5f$, they cannot when $n \leq 5f$. Thus optimal resilience, $n > 3f$, cannot be achieved within the canonical-round framework.

On the positive side, we show that the gather primitive captures the content-dependent communication needed to bypass this limitation. We use gather to obtain a simple and modular algorithm for connected consensus with optimal resilience, clarifying the communication structures required for optimal resilience.

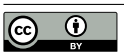
CCS Concepts

• **Theory of computation → Distributed algorithms**.

Keywords

randomized consensus, approximate agreement, crusader agreement, connected consensus, reliable broadcast, gather

ACM Reference Format:
Hagit Attiya, Itay Flam, and Jennifer L. Welch. 2026. Why Canonical-Round Algorithms Fail for Optimal Byzantine Resilience. In *ACM Symposium on Principles of Distributed Computing (PODC '26), July 06–10, 2026, Egham, United Kingdom.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3796701.3815948



This work is licensed under a Creative Commons Attribution 4.0 International License.
PODC '26, Egham, United Kingdom
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2512-8/2026/07
https://doi.org/10.1145/3796701.3815948

1 Introduction

Consensus is the fundamental coordination problem in distributed computing. In asynchronous systems with Byzantine failures, deterministic consensus is impossible, but randomized consensus is solvable with optimal resilience $n > 3f$, where n is the number of processes and f is the maximum number of faulty processes. Randomized Byzantine consensus under optimal resilience lies at the core of many fault-tolerant systems, including modern blockchain protocols.

Ben-Or's pioneering randomized consensus algorithm [11] uses a simple communication pattern called *canonical (asynchronous) rounds* [25], in which processes repeatedly exchange round-tagged messages, wait for $n - f$ messages from the current round, and then advance. Besides its conceptual simplicity, counting the number of asynchronous rounds is a natural way to analyze performance. However, Ben-Or's algorithm requires $n > 5f$ and takes an exponential expected number of rounds, both of which are sub-optimal.

Work on developing improved algorithms for randomized consensus continues to this day, with much emphasis on achieving optimal resilience of $n > 3f$. These algorithms are sophisticated, and their communication patterns are more involved than that in [11]. For instance, Canetti and Rabin's algorithm [15] runs in constant expected time and requires $n > 3f$, but is an intricate combination of primitives. One of these primitives is Bracha's *reliable broadcast* algorithm [13], which prevents faulty processes from equivocating by ensuring that correct processes agree on the value sent by a given sender. This algorithm, reproduced as Algorithm 1, relies on content-dependent wait conditions that are not satisfied by merely advancing through canonical rounds. For example, Step 1 is not complete until various numbers of various kinds of messages have been received.

This raises a natural question: *can optimal-resilience randomized consensus be achieved within the simple structure of canonical rounds?* A strawman attempt to translate Algorithm 1 into canonical rounds is shown in Algorithm 2. Unfortunately, it can experience an unbounded number of rounds: Even with a correct sender, the sender's *initial* message can be delayed arbitrarily long, while canonical rounds consisting largely of *null* messages continue to advance. In contrast, the *time complexity* of this algorithm is constant, when measured as the real time elapsed until termination assuming every message sent between correct processes is delivered within one time unit [9]: Though Byzantine processes may "rush" correct processes through many canonical rounds, as long as the sender's *initial* message is in transit, less than one time unit can elapse. Thus canonical round complexity can diverge arbitrarily

from real time: the algorithm terminates in constant time while requiring an unbounded number of canonical rounds.

This bad behavior is not simply an artifact of poor translation. We identify a fundamental limitation of canonical round-based abstractions in the Byzantine setting. *When $3f < n \leq 5f$, no randomized canonical-round algorithm can solve Byzantine consensus with bounded expected round complexity. Moreover, if communication closure is imposed, which requires early and late messages to be discarded, canonical-round algorithms fail to solve consensus altogether in this resilience regime.*

These results are not restricted to randomized consensus. Our theorems are proved for a generalization of consensus that we call *nontrivial convergence*. This class of problems includes *approximate agreement* (both on the real numbers and on graphs) and *connected consensus* [10] (including its special cases *crusader agreement* and *gradecast*). Unlike consensus, these problems have deterministic solutions in asynchronous systems. The deterministic versions of our lower bounds imply that no optimally-resilient canonical-round algorithm for any of these problems can have bounded worst-case round complexity, and no communication-closed algorithms, in which messages from earlier rounds are ignored, exist.

We extend these limitations to two communication primitives, reliable broadcast and gather, via reductions from crusader agreement. The reduction to reliable broadcast justifies the poor behavior of Algorithm 2. Canonical rounds enforce progress based solely on *message counts*, whereas optimally-resilient Byzantine algorithms require progress conditions that depend on *message contents*. This is exemplified with *gather*, which captures a form of content-dependent communication that canonical rounds exclude: informally, gather allows each process to collect a large set of values such that all correct processes share a substantial *common core*. There are existing gather implementations with optimal resilience and constant time complexity [32] (see [8]).

We extend the reduction used to establish the impossibility results for gather into a simple and modular algorithm for connected consensus for any parameter R, yielding an optimally-resilient algorithm whose running time is logarithmic in R. An additional advantage of this reduction is that it preserves the *binding* property [4]: once the first correct process decides, the adversary cannot influence the decisions of other correct processes. When gather satisfies binding, the resulting connected-consensus algorithm inherits this property directly.

In summary, this paper makes the following contributions relating to asynchronous systems subject to Byzantine failures.

- We show that no randomized canonical-round algorithm can solve a nontrivial convergence problem with bounded expected round complexity when $3f < n \leq 5f$, and that communication-closed canonical-round algorithms fail completely in this regime. *Thus there can be no such algorithms for randomized consensus that have optimal resilience $n > 3f$.*
- The deterministic versions of our lower bounds imply that no canonical-round algorithms have bounded worst-case round complexity, and no communication-closed canonical-round algorithms exist at all, for approximate agreement and R-connected consensus that have optimal resilience $n > 3f$.
- Simple reductions from crusader agreement extend our lower bounds to two popular communication primitives, gather and reliable broadcast.
- The reduction from crusader agreement to gather forms the basis of a modular R-connected consensus algorithm, for any $R \geq 1$, that has optimal resilience and preserves the binding property. Beyond highlighting the usefulness of gather, the resulting algorithm is of independent interest.

Algorithm 1 Reliable Broadcast ($n > 3f$), from [13]

1: **Step 0 (by the sender):** send $(initial, v)$ to all processes ▷ triggered by r-broadcast(v)
2: wait until the receipt of: ▷ **Step 1**
   one $(initial, v)$ message or $(n+f)/2$ $(echo, v)$ messages
   or $f+1$ $(ready, v)$ messages for some v
3: send $(echo, v)$ to all the processes.
4: wait until the receipt of: ▷ **Step 2**
   $(n+f)/2$ $(echo, v)$ messages or $f+1$ $(ready, v)$ messages
   (including ones received in step 1) for some v
5: send $(ready, v)$ to all the processes
6: wait until the receipt of: ▷ **Step 3**
   $2f+1$ $(ready, v)$ messages
   (including ones received in steps 1 and 2) for some v
7: **r-accept** v

Algorithm 2 Canonical Round Reliable Broadcast ($n > 3f$), based on [13]; code for process p_i

local variable *round*, initially 1
1: **WakeUp:** ▷ triggered by r-broadcast(v) at the sender ▷ and r-broadcast($-$) at other processes
2: **if** p_i is the sender **then**
   send $(initial, v)$ labeled with round 1 to all processes
3: **else** send($null$) labeled with round 1 to all processes **endif**
4: **Receive message** m **with round label** r**:**
5: **if** $r = round$ and p_i has now received $n - f$ round r messages **then**
6: $S := \emptyset$ ▷ set of messages to send for round $r+1$
7: **if** p_i has received $(initial, v)$ or $(n+f)/2$ $(echo, v)$'s or $f+1$ $(ready, v)$'s and not sent $(echo, v)$ **then** add $(echo, v)$ to S **endif**
8: **if** p_i has received $(n+f)/2$ $(echo, v)$'s or $f+1$ $(ready, v)$'s and not sent $(ready, v)$ **then** add $(ready, v)$ to S **endif**
9: **if** p_i has received $2f+1$ $(ready, v)$'s and not already done r-accept **then r-accept** v **endif**
10: $round := round + 1$
11: **if** $S = \emptyset$ **then** $S := (null)$ **endif**
12: send messages in S labeled with current value of *round* variable to all processes
endif

Related Work

Round-based abstractions, such as canonical rounds [25], the *Heard-Of model* [16] and communication-closed layers [24], make asynchronous executions appear synchronous which enables modular reasoning and intuitive running time calculations. In the canonical round structure, processes must move to the next round as soon as a fixed *number* of messages have been received, regardless of the message contents or the identities of the senders. Several well-known

Byzantine-tolerant algorithms are structured in canonical rounds, including ones for randomized consensus [11], approximate agreement [22], and crusader agreement [10], albeit assuming $n > 5f$. In contrast, many optimally resilient Byzantine algorithms rely critically on content-dependent communication patterns, including those for randomized consensus [15], crusader agreement [3, 4, 21] and its generalization to connected consensus [10], approximate agreement [2, 18], gather [15, 32], and reliable broadcast [13]. Our results explain the dichotomy between algorithms that require $n > 5f$ and those that achieve optimal resilience $n > 3f$.

Round-based abstractions have played an important role in structuring, analyzing, and verifying distributed algorithms, and have been extended to Byzantine transmission faults [12]. These abstractions are particularly effective for benign fault models [6] and for systematic testing and verification [20, 23]. Our impossibility results clarify their limitations in the Byzantine setting: while useful for reasoning and verification, they cannot capture the communication patterns required for optimal-resilience Byzantine coordination.

In contrast to some early work which gave the impression implicitly that asynchronous algorithms could be structured as canonical rounds without any loss of generality (e.g., [17, 22, 25]), later authors indicated some skepticism on this point (e.g., [16]). Lewko [28] in particular notes that it "is also worth considering if the seemingly natural notion of a round (imported from the synchronous setting) may have a restrictive effect on our thinking in the asynchronous setting." Our results show that the notion of round is indeed restrictive when optimal Byzantine resilience is desired.

A preliminary version of this paper appeared in [7].

2 Preliminaries

In this section, we present our model of computation. We also define a generic problem, called "nontrivial convergence", and show that several well-known problems are special cases of it. We then define canonical-round algorithms and present our complexity measures.

2.1 Model of Computation

We assume the standard asynchronous model for n processes, up to f of which can be faulty, in which processes communicate via reliable point-to-point messages. We consider *Byzantine* failures, where a faulty process can change state arbitrarily and send messages with arbitrary content.

In more detail, we assume a set of n processes, each modeled as a state machine. Each process has a subset of initial states, with one state corresponding to each element of a set V, denoting its input. The transitions of the state machine are triggered by two kinds of *events*: spontaneous WakeUp and receipt of a message. A transition takes the current state of the process, a random number, and an incoming message (if any), and produces a new state of the process and a set of messages to be sent to any subset of the processes. The state set of a process contains a collection of disjoint subsets, each one modeling the fact that a particular decision has been taken; once a process enters the subset of states for a specific decision, the transition function ensures that it never leaves that subset.

A *configuration* of the system is a vector of process states, one for each process, and a set of in-transit messages. In an *initial configuration*, each process is in an initial state and no messages are in transit. Given a subset of at most f processes that are *faulty* with the rest being *correct*, we define an *execution* as a sequence α of alternating configurations and events $C_0, e_1, C_1, \ldots$ such that:

- C_0 is an initial configuration.
- The first event for each process is WakeUp. A correct process experiences exactly one WakeUp and a faulty process can experience any number of WakeUps. The WakeUp can either be spontaneous (e.g., triggered by the invocation of the algorithm) or in response to the receipt of a message.
- Suppose e_i is an event in which process p receives message m sent by process q. Then m is an element of the set of in-transit messages in C_{i-1} and it is the oldest in-transit message sent by q to p. This means that point-to-point links are FIFO.
- Suppose e_i is a step by correct process p and let s and M be the state and set of messages resulting from p's transition function applied to p's state in C_i, a random number, and, if e_i is a receive event, the message m being received. Then the only differences between C_i and C_{i+1} are that in C_{i+1}, m is no longer in transit, M is in transit, and p's state is s. If p is Byzantine, then s and M can be anything.
- If α is infinite, then every correct process takes infinitely many steps and every message sent by a process to a correct process is eventually received.

We model the variability in asynchronous executions, other than that caused by random numbers, by an *adversary*, which is a function that takes the current prefix of the execution and returns the next event to occur. The adversary is subject to the constraints indicated above. The adversary is *full information* as it can see the random choices made so far by the processes which are reflected in the process states and message contents, and *static* as it chooses the faulty processes initially.

We say that executions α and β are *indistinguishable* to a set of processes X, denoted $\alpha \overset{X}{\sim} \beta$, if, for each process p in X, p has the same initial state and experiences the same sequence of events and random numbers in α as in β.

To measure time complexity in an asynchronous message-passing system, we adopt the definition in [9]: We start by defining a timed execution as an execution in which nondecreasing nonnegative integers ("times") are assigned to the events, with no two events by the same process having the same time. For each timed execution, we consider the prefix ending when the last correct process decides, and then scale the times so that the *maximum time* that elapses between the sending and receipt of any message between correct processes is 1. We define the time complexity as the maximum, over all such scaled timed execution prefixes, of the time assigned to the last event minus the latest time when any (correct) process wakes up. We sometimes assume, for simplicity, that the first WakeUp event of each process occurs at time 0.

2.2 Nontrivial Convergence Problems

Our lower bound results are proved for a generic *nontrivial convergence* problem in which there are at least two possible input values x_0 and x_1 and at least two decision values d_0 and d_1, such that:

- **Agreement:** if a correct process decides d_0 in an execution, then no correct process can decide d_1 in the same execution.

- **Validity:** if all correct processes have input x_i, then every decision by a correct process is d_i, for $i = 0, 1$.

For *deterministic* algorithms, we also require:

- **Termination:** All correct processes eventually decide.

When considering *randomized* algorithms, Agreement and Validity must always hold, but Termination is probabilistic. See Section 2.3 for details.

Consensus [31] is clearly a nontrivial convergence problem. In addition to termination, all decisions by correct processes must be the same (*agreement*), and if a correct process decides v, then some correct process has input v (*validity*). Several well-known relaxations of consensus also belong in this class, as we show next.

Approximate agreement on the real numbers with parameter $\epsilon > 0$ [22] is defined as follows. Processes start with arbitrary real numbers and correct processes must decide on real numbers that are at most ϵ apart from each other (*agreement*). Decisions must also be contained in the interval of the inputs of correct processes (*validity*). To show approximate agreement is a nontrivial convergence problem, choose any two real numbers whose difference is greater than ϵ as the two distinguished inputs and two distinguished decisions.

Approximate agreement on graphs [30] has each process start with a vertex of a graph G as its input. Correct processes must decide on vertices such that all decisions are within distance one of each other (*agreement*) and inside the convex hull of the inputs (*validity*). When all processes start with the same vertex, validity implies they must decide on this vertex. As long as the graph G has two vertices that are at distance 2 apart, we can choose these vertices as the two distinguished inputs and two distinguished decisions to show that approximate agreement on G is a nontrivial convergence problem.

Connected consensus [10] with parameter R and input set V ensures that correct processes decide on elements in $D = \{(v, g) | v \in V \text{ and } 1 \leq g \leq R\} \cup \{\perp, 0\}$; we refer to v as the *branch* and g as the *grade*. If two correct processes decide (v, g) and (v', g'), then $|g - g'| \leq 1$ and $v = v'$ unless one is $\perp$ (*agreement*). In addition, if $v \neq \perp$, then some correct process has input v, and if all correct processes have the same input, then $g = R$ (*validity*). To show that this is a nontrivial convergence problem, let v and v' in V be the distinguished inputs and (v, R) and (v', R) be the distinguished decisions. When $R = 1$, we get **crusader agreement** [21]. When $R = 2$, we get **gradecast** [26].

2.3 Canonical-Round Algorithms and Their Round Complexity

In a canonical-round algorithm, every message sent by a correct process consists of a round number (a positive integer), the process' history (its initial state followed by the sequence of random numbers and sequence of messages it has received so far), and a random number drawn from a set $\mathcal{R}$. For deterministic algorithms, $|\mathcal{R}| = 1$ and thus we can drop that component of the messages.

Each correct process wakes up and sends to all processes its round 1 message, consisting of the label 1, its initial state, and (optionally) a random number. This step is called round 0 for the process. All messages that arrive at a process are saved, including those that arrive before it wakes up. Once a process has received $n - f$ round r messages, where r is the round number of the most recent message it sent, it sends its round $r + 1$ message to all processes. The interval starting immediately after a process sends its round r message and ending when it sends its round $r + 1$ message is called round r for the process. In each round, a process applies a function to its current history to see if it can decide yet. Even if it decides, it continues to send messages, in order to help other processes make progress toward decision.

Given an execution of a canonical-round algorithm, we define the (canonical) round complexity of the execution to be the smallest round r such that every correct process decides in or before its round r. The *round complexity* of a deterministic algorithm is the largest round complexity of any execution of the algorithm.

We next define the expected round complexity of a randomized algorithm (cf. [9]). An execution of a specific algorithm is uniquely determined by an adversary $\mathcal{A}$, an initial configuration I, and a collection of random numbers ρ accessed by the processes, denoted $exec(\mathcal{A}, I, \rho)$. Given a predicate P on executions, $\Pr[\mathcal{A}, I, P]$ is the probability of $\{\rho : exec(\mathcal{A}, I, \rho) \text{ satisfies } P\}$, i.e., the probability taken over the random numbers that the execution produced by $\mathcal{A}$ starting with I satisfies P. Let T be a random variable on executions, in our case, the round complexity. For a fixed $\mathcal{A}$ and I, the expected value of T, denoted $EX[\mathcal{A}, I, T]$, is $\sum_x x \cdot \Pr[\mathcal{A}, I, T = x]$. The *expected round complexity* of the algorithm, denoted $EX[T]$, is the maximum, over all adversaries $\mathcal{A}$ and all initial configurations I, of $EX[\mathcal{A}, I, T]$.

3 Limitations of Canonical-Round Algorithms

This section contains our lower bound results for canonical-round algorithms concerning the number of rounds (Section 3.1) and impossibility when communication-closed (Section 3.2).

3.1 Unbounded Number of Canonical Rounds

Theorem 1. *For any randomized canonical-round algorithm that solves the nontrivial convergence problem with $n \leq 5f$, the expected number of rounds until all correct processes decide is greater than K, for any integer $K \in \mathbb{N}$.*

Proof. Assume towards a contradiction that there exists a canonical-round algorithm for nontrivial convergence with $n \leq 5f$ whose expected number of rounds until all correct processes decide is at most K. For convenience, denote the specific values x_0, x_1, d_0, and d_1 in the definition of nontrivial convergence by 0, 1, 0, and 1 respectively.

For simplicity, we assume $n = 5f$, and divide the processes into five disjoint sets of f processes each: A, B, C, D, and E. We abuse notation and, for any two sets X and Y, denote $X \cup Y$ by XY.

We consider the following initial configurations of the algorithm:

- Denote by C_0 the initial configuration in which processes in $BCDE$ are correct and have input 0, while processes in A are Byzantine.
- Denote by C_1 the initial configuration in which processes in $ABDE$ are correct and have input 1, while processes in C are Byzantine.

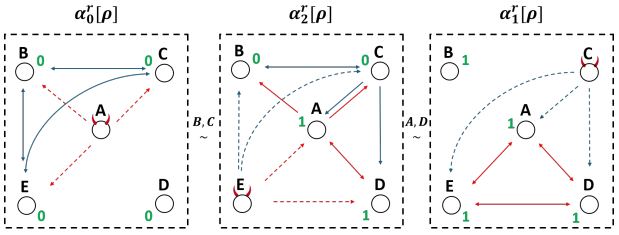


Figure 1: Schematic description of the executions used in the proof of Theorem 1. Messages are represented by directed arrows, with dotted arrows being those sent by Byzantine processes. Blue arrows correspond to 0 inputs and red to 1. Note two-faced behavior of E in $\alpha_2^r[\rho]$.

- Denote by C_2 the initial configuration in which processes in $ABCD$ are correct, those in BC have input 0 and those in AD have input 1, while processes in E are Byzantine.

For each C_j, let $C_j[i]$ denote p_i's input in C_j.

We next define a set of executions of the algorithm that begin with C_2. For each of these executions, we define a related execution that begins with C_0 and another one that begins with C_1. All these executions are inductively constructed, round by round, with the behavior of the faulty processes in each one being dependent on the behavior of some correct processes in others. (See Figure 1.)

The construction uses a $(K+1) \times 5f$ array of elements from $\mathcal{R}$, denoted ρ, where $\mathcal{R}$ is the set from which the random numbers are chosen. For each r and i, $\rho[r, i]$ contains the random choice made by process p_i in round r of certain executions.

Basis step of the construction: Define execution $\alpha_2^0[\rho]$, in which processes wake up and send their round 1 messages, as follows:

- Start with C_2.
- $ABCDE$ wake up.
- Each (correct) $p_i \in ABCD$ sends $(1, C_2[i], \rho[1, i])$ to $ABCDE$.
- Each (faulty) $p_i \in E$ sends $(1, C_0[i], \rho[1, i])$ to BC and $(1, C_1[i], \rho[1, i])$ to AD. That is, p_i pretends to BC that its input is 0 and pretends to AD that its input is 1.

Next we construct execution $\alpha_0^0[\rho]$ as follows:

- Start with C_0.
- $ABCE$ wake up. (Processes in D are late.)
- Each (correct) $p_i \in BCE$ sends $(1, C_0[i], \rho[1, i])$ to $ABCDE$.
- Each (faulty) $p_i \in A$ sends $(1, C_2[i], \rho[1, i])$ to $ABCDE$. That is, p_i pretends that it is correct in $\alpha_2^0[\rho]$.

Similarly, we construct execution $\alpha_1^0[\rho]$ as follows:

- Start with C_1.
- $ACDE$ wake up. (Processes in B are late.)
- Each (correct) $p_i \in ADE$ sends $(1, C_1[i], \rho[1, i])$ to $ABCDE$.
- Each (faulty) $p_i \in C$ sends $(1, C_2[i], \rho[1, i])$ to $ABCDE$. That is, p_i pretends that it is correct in $\alpha_2^0[\rho]$.

Inductive step of the construction: For r, $1 \le r \le K$, assume that executions $\alpha_i^{r-1}[\rho]$, $i = 0, 1, 2$, have already been defined.

Define execution $\alpha_2^r[\rho]$, in which processes receive round r messages and send round $r+1$ messages, as follows:

- Start with $\alpha_2^{r-1}[\rho]$.
- AD receive the round r messages which are in transit at the end of $\alpha_2^{r-1}[\rho]$ from $ACDE$.
- BC receive the round r messages which are in transit at the end of $\alpha_2^{r-1}[\rho]$ from $ABCE$.
- Each (correct) $p_i \in AD$ sends $(r+1, h_i, \rho[r+1, i])$ to $ABCDE$, where h_i is p_i's history at the end of $\alpha_2^{r-1}[\rho]$ followed by the round r messages just received from $ACDE$.
- Each (correct) $p_i \in BC$ sends $(r+1, h_i, \rho[r+1, i])$ to $ABCDE$, where h_i is p_i's history at the end of $\alpha_2^{r-1}[\rho]$ followed by the round r messages just received from $ABCE$.
- Each (faulty) $p_i \in E$ sends $(r+1, h_i^0, \rho[r+1, i])$ to BC, where h_i^0 is the state of p_i at the end of $\alpha_0^{r-1}[\rho]$ followed by the round r messages in transit from $ABCE$ to p_i at the end of $\alpha_0^{r-1}[\rho]$.
- Each (faulty) $p_i \in E$ sends $(r+1, h_i^1, \rho[r+1, i])$ to AD, where h_i^1 is the state of p_i at the end of $\alpha_1^{r-1}[\rho]$ followed by the round r messages in transit from $ACDE$ to p_i at the end of $\alpha_1^{r-1}[\rho]$.

That is, each faulty process in E pretends to BC that it is correct in $\alpha_0^r[\rho]$ and pretends to AD that it is correct in $\alpha_1^r[\rho]$.

Observation 2. *Every process completes r canonical rounds in $\alpha_2^r[\rho]$.*

Next we construct execution $\alpha_0^r[\rho]$ as follows:

- Start with $\alpha_0^{r-1}[\rho]$.
- $ABCE$ receive round r messages which are in transit at the end of $\alpha_0^{r-1}[\rho]$ from $ABCE$.
- Each (correct) $p_i \in BCE$ sends $(r+1, h_i, \rho[r+1, i])$ to $ABCDE$, where h_i is p_i's history at the end of $\alpha_0^{r-1}[\rho]$ followed by the round r messages it just received from $ABCE$.
- Each (faulty) $p_i \in A$ sends $(r+1, h_i, \rho[r+1, i])$ to $ABCDE$, where h_i is p_i's history at the end of $\alpha_2^{r-1}[\rho]$ followed by the round r messages in transit from $ACDE$ to p_i at the end of $\alpha_2^{r-1}[\rho]$. That is, p_i pretends that it is correct in $\alpha_2^r[\rho]$.

Similarly, we construct execution $\alpha_1^r[\rho]$ as follows:

- Start with $\alpha_1^{r-1}[\rho]$.
- $ACDE$ receive round r messages which are in transit at the end of $\alpha_1^{r-1}[\rho]$ from $ACDE$.
- Each (correct) $p_i \in ADE$ sends $(r+1, h_i, \rho[r+1, i])$ to $ABCDE$, where h_i is p_i's history at the end of $\alpha_1^{r-1}[\rho]$ followed by the round r messages it just received from $ACDE$.
- Each (faulty) $p_i \in C$ sends $(r+1, h_i, \rho[r+1, i])$ to $ABCDE$, where h_i is p_i's history at the end of $\alpha_2^{r-1}[\rho]$ followed by the round r messages in transit from $ABCE$ to p_i at the end of $\alpha_2^{r-1}[\rho]$. That is, p_i pretends that it is correct in $\alpha_2^r[\rho]$.

The next lemma is proved by induction on the rounds.

Lemma 3. *For all r, $0 \le r \le K$, $\alpha_2^r[\rho] \overset{BC}{\sim} \alpha_0^r[\rho]$ and the same set of messages are in transit from $ABCE$ to BC at the end of both executions.*

Proof. By induction on r. Note that BC are correct in both executions.

Basis: $r = 0$. Each $p_i \in BC$ has the same input in C_2 as in C_0. In both executions, all p_i does is wake up and send a round 1 message with the same content. Thus $\alpha_2^0[\rho] \overset{BC}{\sim} \alpha_0^0[\rho]$.

We just argued that processes in BC send the same messages in the two executions. Each $p_i \in AE$ sends the same message in

both executions by definition: If $p_i \in A$, then its faulty behavior in $\alpha_0^0[\rho]$ is to send the same message that it sends in $\alpha_2^0[\rho]$ where it is correct. If $p_i \in E$, then its faulty behavior in $\alpha_2^0[\rho]$ is to send the same message that it sends in $\alpha_0^0[\rho]$ where it is correct.
Inductive step: $r \geq 1$. Assume the lemma is true for $r-1$.

By the inductive hypothesis, each $p_i \in BC$ starts its round r with the same history in both $\alpha_2^r[\rho]$ and $\alpha_0^r[\rho]$. By the inductive hypothesis, the same set of round r messages are in transit to p_i from *ABCE* at the beginning of p_i's round r in both executions. Since by construction, p_i receives round r messages from *ABCE* during its round r in both executions, it has the same history at the end of its round r in both executions. Thus $\alpha_2^r[\rho] \overset{BC}{\sim} \alpha_0^r[\rho]$.

The argument just made also implies that each process in *BC* sends the same round $r+1$ messages to *BC* in the two executions. Each $p_i \in A$ sends the same round $r+1$ messages to *BC* in the two executions because its faulty behavior in $\alpha_0^r[\rho]$ is to send the same message that it sends in $\alpha_2^r[\rho]$ where it is correct. Each $p_i \in E$ sends the same round $r+1$ messages to *BC* in the two executions because its faulty behavior in $\alpha_2^r[\rho]$ is to send to *BC* the same message that it sends in $\alpha_0^r[\rho]$ where it is correct. □

A similar argument proves the next lemma.

Lemma 4. *For all r, $0 \leq r \leq K$, $\alpha_2^r[\rho] \overset{AD}{\sim} \alpha_1^r[\rho]$ and the same set of messages are in transit from ACDE to AD at the end of both executions.*

Let $E_2^K = \{\alpha_2^K[\rho] | \rho \text{ is any } (K+1) \times 5f \text{ array of elements from } \mathcal{R}\}$.

Lemma 5. *There exists an execution in E_2^K in which all correct processes decide.*

Proof. The key point is to show that E_2^K consists of all the K-round executions, starting from C_2, that can be generated by a fixed adversary $\mathcal{A}$. The construction of $\alpha_2^K[\rho]$ just given for a fixed collection of random numbers ρ describes the function $\mathcal{A}$ specifying the order of process steps and message delays up to the point when all processes have completed K rounds. (The execution can be extended infinitely in any arbitrary way as long as every correct process takes an infinite number of steps and all messages to correct processes are eventually received, but we only care about the execution through K rounds.) Thus E_2^K consists of all K-round executions that have the same scheduling as that described for $\mathcal{A}$ but with all the possible values for the collection ρ of random numbers used by the processes.

Since the expected round complexity $EX[T]$ is assumed to be at most K, we have:

$$K \geq EX[T] \geq EX[\mathcal{A}, C_2, T] = \sum_{x \in \mathbb{N}} x \cdot \Pr[\mathcal{A}, C_2, T = x] \ .$$

Thus there is at least one value of $x \leq K$ such that $\Pr[\mathcal{A}, C_2, T = x]$ is positive. Let ρ be such that $exec(\mathcal{A}, C_2, \rho)$ has round complexity x. The execution $exec(\mathcal{A}, C_2, \rho)$ is the same as $\alpha_2^K[\rho]$, and in this execution all correct processes decide by round K. □

Let $\alpha_2^K[\rho]$ be an execution whose existence is guaranteed by Lemma 5.

By Lemma 3 with $r = K$, $\alpha_2^K[\rho] \overset{BC}{\sim} \alpha_0^K[\rho]$ and thus every process p_i in *BC* decides by the end of $\alpha_0^K[\rho]$. By the validity condition, p_i decides 0 in $\alpha_0^K[\rho]$, and thus it also decides 0 in $\alpha_2^K[\rho]$. Similarly, by Lemma 4 with $r = K$, $\alpha_2^K[\rho] \overset{AD}{\sim} \alpha_1^K[\rho]$ and thus every process p_i in *AD* decides by the end of $\alpha_1^K[\rho]$. By the validity condition, p_i decides 1 in $\alpha_1^K[\rho]$, and thus it also decides 1 in $\alpha_2^K[\rho]$.

But $\alpha_2^K[\rho]$ violates the agreement condition, a contradiction. □

We have the following corollary for deterministic algorithms:

Corollary 6. *For any deterministic canonical-round algorithm that solves the nontrivial convergence problem with $n \leq 5f$, the worst-case number of rounds until all correct processes decide is greater than K, for any integer $K \in \mathbb{N}$.*

Proof. The proof is essentially the same as that of Theorem 1 without the need for ρ (the random choices). The key difference is that finding an execution in E_2^K in which all correct processes decide (cf. Lemma 5) is much simpler. E_2^K contains a single execution and by the contradiction assumption that the worst-case round complexity is at most K, every correct process decides in it. □

3.2 Impossibility with Communication-Closed Canonical Rounds

We model algorithms with communication-closed canonical rounds as follows: processes proceed in canonical rounds, but round r messages that arrive in a later round than r are discarded. As before, processes keep sending messages after they decide, and never halt.

Theorem 1 does not rule out the possibility that the expected number of rounds for each adversary is finite, but there is no upper bound that holds for all adversaries. Under the additional constraint of communication-closed behavior, we can prove something stronger, that there is an adversary that can prevent decision in all its executions.

Theorem 7. *For any randomized communication-closed canonical-round algorithm that solves the nontrivial convergence problem with $n \leq 5f$, there exists an adversary $\mathcal{A}$ and initial configuration I such that $\Pr[\mathcal{A}, I, P] = 0$, where P is the predicate that all the correct processes eventually decide.*

The proof is very similar to that of Theorem 1 and also works by contradiction. We define a similar adversary $\mathcal{A}$ to the one in the previous proof. The key differences are:

- The executions constructed are extended to infinitely many rounds, not just K.
- The processes that were late to wake up in the previous proof wake up initially but have their messages delivered one round late.
- All the messages that are delayed until after round K in the original executions are delivered one round late.

To obtain an execution $\alpha_2^K[\rho]$ in which all correct processes decide, we argue that by the contradiction assumption, there is some set of random choices ρ such that in the execution from C_2 scheduled by $\mathcal{A}$ and using ρ, all correct processes decide. Let K be the round by which the decisions are made. The rest of the proof follows as for Theorem 1; see [8].

We have the following corollary for deterministic algorithms:

Corollary 8. *It is impossible for any deterministic communication-closed canonical-round algorithm to solve the nontrivial convergence problem with $n \leq 5f$.*

Proof. Assume in contradiction there is such an algorithm. Consider the adversary $\mathcal{A}$ and initial configuration I from the statement of Theorem 7. When applied to the deterministic algorithm, there is only one resulting execution and it does not satisfy the predicate that all the correct processes eventually decide. □

3.3 Immediate Consequences

We now examine the consequences of Theorems 1 and 7 and their deterministic corollaries for several fundamental tasks in asynchronous Byzantine-tolerant systems. We focus on deterministic solutions for all the tasks considered, except for consensus which has only randomized solutions.

Recall from Section 2.3 that **consensus**, **ϵ-approximate agreement** (where the input set contains two values more than ϵ apart), **approximate agreement on a graph** (with two vertices at distance 2), and **R-connected consensus** (which generalizes **crusader agreement** and **gradecast**) are all examples of nontrivial convergence problems.

Thus Theorem 1 implies that for each of these problems, the expected round complexity of any *randomized* canonical-round algorithm with $n \leq 5f$ is unbounded. Furthermore, by Theorem 7 such an algorithm cannot be communication closed.

In addition, Corollary 6 implies that any *deterministic* canonical-round algorithm for each of these problems with $n \leq 5f$ has an execution in which some correct process does not decide by round K, for any $K \in \mathbb{N}$. Also, by Corollary 8, such an algorithm cannot be communication closed.

4 Canonical-Round Lower Bounds via Reductions

In this section we use reductions to apply our lower bounds from Section 3 beyond the realm of nontrivial convergence problems. We show that crusader agreement (i.e., connected consensus with $R = 1$) can be used in a reduction to show that reliable broadcast is subject to the same limitations. Crusader agreement can also be used to solve a problem called *gather* with no additional communication, and thus the limitations on crusader agreement also apply to gather.

4.1 Reliable and Consistent Broadcast

Reliable broadcast [13] is defined with one of the n processes, s, designated as the *sender*. The sender has an input value v, and it calls *r-broadcast*(v, s), where the argument s indicates that s is the sender in this instantiation. Processes other than p call *r-broadcast*$(-, s)$, where the argument $-$ indicates that the invoker is not the sender in this instantiation. Processes may terminate with *r-accept*(w,s), with the following properties:

Agreement: All correct processes that accept a value from sender s, accept the same value.

Validity: If the sender s is correct then eventually all correct processes accept s's input.

Algorithm 3 Crusader agreement using consistent broadcast ($n > 4f$); code for process p_i with input v_i

```
1: W_i ← ∅                                          ▷ W_i is a multiset of values
2: c-broadcast(v_i, p_i)                            ▷ invoke c-broadcast as sender
3: c-broadcast(−,p_j) for all j ≠ i
                                    ▷ invoke n − 1 c-broadcasts as non-sender
4: repeat
     upon c-accept(v, p_j): W_i ← W_i ∪ {v}
5: until |W_i| = n − f
6: if W_i contains |W_i| − f copies of v then decide v
7:    else decide ⊥ endif
```

Totality (relay): If some correct process accepts a value from sender s then eventually all correct processes accept a value from sender s.

A weaker variant, *consistent broadcast* [14], is very similar but abandons the totality property; it requires only that correct processes do not accept conflicting values from the same sender, and that if the sender is correct then eventually all correct processes accept its message. The interface to consistent broadcast is denoted c-broadcast and c-accept.

We use a reduction to show that consistent broadcast has no bounded-round canonical round algorithm and no communication-closed canonical round algorithm when $n \leq 5f$. Since every algorithm for reliable broadcast is also a solution to consistent broadcast, the impossibility results for consistent broadcast immediately extend to reliable broadcast. Consider Algorithm 3 for crusader agreement, assuming $n > 4f$, which uses n concurrent instantiations of consistent broadcast. Next we show that this algorithm is correct.

Theorem 9. *Algorithm 3 solves crusader agreement for $n > 4f$.*

Proof. To argue *agreement* for crusader agreement, assume for contradiction that a correct process p_i has $|W_i| - f$ copies of v in W_i, and a correct process p_j has $|W_j| - f$ copies of w in W_j. Then, since $|W_i|, |W_j| \geq n - f$ and since $n > 4f$, p_i has c-accepted v from some process, while p_j has c-accepted w from the same process, in contradiction to the agreement property of consistent broadcast.

To argue *validity* for crusader agreement, it is clear that when all correct processes start with v, each correct process will c-accept at least $|W_i| - f$ copies of v and thus decide v.

To show *termination* for crusader agreement, note that Algorithm 3 simply waits for the termination of $n-f$ out of n concurrent invocations of consistent broadcast. □

Since Algorithm 3 adds no communication beyond that of the n copies of consistent broadcast that run in parallel, if the consistent broadcast is a (communication-closed) canonical round algorithm, then so is Algorithm 3. This contradicts the results of Section 3.3, and implies the following corollary, which justifies why Algorithm 1 [13] is not in canonical round form:

Corollary 10. *In the asynchronous model with $n \leq 5f$, any canonical round algorithm for reliable or consistent broadcast has an execution in which some correct process does not terminate by round K, for any integer $K \geq 1$. Furthermore, such an algorithm is not communication-closed.*

Algorithm 4 Modular Byzantine-tolerant R-connected consensus for arbitrary $R \geq 1$ ($n > 3f$); code for process p_i. Uses subroutines G for gather and AA for approximate agreement.

1: $S_i \leftarrow \mathrm{G}(x_i)$ ▷ call gather subroutine; x_i is p_i's input
2: **if** some $v \in V$ appears $|S_i| - f$ times in S_i **then** $v_i \leftarrow v$; $g_i \leftarrow 1$ **else** $v_i \leftarrow \perp$; $g_i \leftarrow 0$ **endif**
3: **if** $R = 1$ **then** decide (v_i, g_i) **endif** ▷ crusader agreement case; return after deciding ▷ rest of code is for $R \geq 2$ case
4: $T_i \leftarrow \mathrm{G}(v_i)$ ▷ call gather subroutine again, now with only one non-$\perp$ value
5: **if** some $v \in V$ appears $f + 1$ times in T_i **then** $v_i \leftarrow v$ **else** $v_i \leftarrow \perp$ **endif** ▷ choose unique branch
6: **if** some $v \in V$ appears $|T_i| - f$ times in T_i **then** $r_i \leftarrow R$ **else** $r_i \leftarrow 0$ **endif** ▷ choose AA input
7: $g_i \leftarrow \lceil \mathrm{AA}(1, r_i) \rceil$ ▷ choose grade via AA with $\epsilon = 1$
8: **if** $g_i = 0$ **then** decide $(\perp, 0)$ **else** decide (v_i, g_i) **endif** ▷ will show $v_i \neq \perp$ when $g_i > 0$

4.2 Gather

Gather's specification (e.g., [32]) can be viewed as an extension of reliable broadcast in which *all processes* broadcast their value, and accept values from a large set of processes. Beyond properties inherited from reliable broadcast, most notably, that if two correct processes accept a value from another process, it is the same value, gather also ensures that there is a *common core* of $n - f$ values that are accepted by all correct processes. In more detail, *gather* is called by process p_i with an input x_i and it returns a set S_i of distinct (process id, value) pairs satisfying the following properties:

Agreement: For any k, if p_i and p_j are correct and $(k, x) \in S_i$ and $(k, x') \in S_j$, then $x = x'$.

Validity: For every pair of correct processes p_i and p_j, if $(j, x) \in S_i$, then $x = x_j$.

Termination: All correct processes eventually return.

Common core: There is a set S^C of size $n-f$ such that $S^C \subseteq S_i$, for every correct process p_i.

We use a reduction from connected consensus with $R = 1$ (i.e., crusader agreement) to gather to show that gather has no bounded-round canonical round algorithm and no communication-closed canonical round algorithm when $n \leq 5f$. We in fact present a connected consensus algorithm for any $R \geq 1$ that uses subroutines for gather and, if $R \geq 2$, for approximate agreement. The algorithm assumes $n > 3f$ and thus relies on subroutines that also work when $n > 3f$. See Algorithm 4 for the pseudocode. Here we focus on Lines 1–3 for the $R = 1$ case; note that this part of the algorithm makes one call to a gather subroutine and does no additional communication.

To prove the correctness of Algorithm 4, we start with three facts about the outputs of any gather primitive. Let S_i be the output of a correct process p_i and S^C be the guaranteed common core.

Lemma 11. *If a value $v \in V$ appears at least $|S_i| - f$ times in S_i, then v appears at least $|S^C| - f$ times in S^C.*

Proof. For any set S of (process-id, value) pairs and any value v, let $\#(S, v)$ be the number of pairs in S containing v. By the hypothesis of the lemma, $\#(S_i, v) \geq |S_i| - f$. Let $T_i = S_i \setminus S^C$ be the subset of S_i that is not in the common core S^C; then $|T_i| = |S_i| - |S^C|$. We have as needed:

$$\#(S^C, v) = \#(S_i, v) - \#(T_i, v) \geq |S_i| - f - (|S_i| - |S^C|) = |S^C| - f. \quad \square$$

Since $|S^C| = n - f$ and $n > 3f$, we also have:

Proposition 12. *At most one value can appear $|S^C| - f$ times in S^C.*

Proposition 13. *If some $v \in V$ appears $|S_i| - f$ times in S_i, then v is the input of some correct process.*

Proof. Since $|S_i| \geq n - f$ and $n > 3f$, at least one of the occurrences of v in S_i is for a correct process p_j. By the validity property of gather, v is p_j's input. $\square$

Theorem 14. *Algorithm 4 solves connected consensus for $R = 1$.*

Proof. *Termination* holds by the assumed correctness, including termination, of the G subroutine.

Validity: Suppose correct process p_i decides (v, g).

(i) If $v \neq \perp$, we must show that some correct process has input v. Since $v \neq \perp$, by Line 2 there are $|S_i| - f$ copies of v in S_i. Proposition 13 gives the result.

(ii) If all correct processes have the same input, we must show $g = 1$. Suppose all the correct processes have the same input, which we know from (i) is v. By the validity property of gather, all the elements in S_i for correct processes, of which there are at least $|S_i| - f$, have value v. Thus p_i sets g_i to 1 in Line 2 and $g = 1$.

Agreement: By Line 2, the only possible decisions are $(\perp, 0)$ and $(v, 1)$ where $v \in V$. Agreement could be violated only if some correct process p_i decides $(u, 1)$ and another correct process p_j decides $(v, 1)$ where $u \neq v$. Since u appears $|S_i| - f$ times in S_i, Lemma 11 implies that u appears $|S^C| - f$ times in S^C, and similarly v appears $|S^C| - f$ times in S^C. By Proposition 12, u must equal v. $\square$

If the gather subroutine G used in Algorithm 4 is a (communication-closed) canonical-round algorithm, then so is Algorithm 4. Since Algorithm 4 does not add any communication on top of the gather subroutine when $R = 1$, the discussion in Section 3.3 regarding crusader agreement implies:

Corollary 15. *In the asynchronous model with $n \leq 5f$, any canonical-round algorithm for gather has an execution in which some correct process does not terminate by round K, for any integer $K \geq 1$. Furthermore, such an algorithm is not communication-closed.*

5 Modular Connected Consensus Algorithm with Optimal Resilience

5.1 Correctness for Arbitrary $R \geq 1$

In order to prove correctness of Algorithm 4 for any value of R greater than 1, we need the following two lemmas. The first one shows that, thanks to the second call to the gather subroutine, the branch chosen in Line 5 is unique. The second one shows that the decisions are well-defined.

Lemma 16. *If $v \in V$ occurs $f + 1$ times in T_i and $u \in V$ appears $f + 1$ times in T_j for not necessarily distinct correct processes p_i and p_j, then $u = v$.*

Proof. At least one occurrence of v in T_i is for a correct process p_k and by the validity property for gather, v is p_k's input to its second call to G on Line 4. By the code (cf. Line 2), v occurs $|S_k| - f$ times in S_k, and by Lemma 11, v occurs $|S^C| - f$ times in S^C.

The same argument shows that u occurs $|S^C| - f$ times in S^C.

Proposition 12 implies that $u = v$. □

Lemma 17. *If $R \geq 2$ and correct process p_i decides (v, g), then $g = 0$ if and only if $v = \perp$.*

Proof. *Case 1:* g_i is set to 0 in Line 7. Then by the code the decision is $(\perp, 0)$ and so $v = \perp$.

Case 2: g_i is set to a positive value in Line 7. By the validity property of approximate agreement, some correct process p_j calls AA with input $r_j = R$. Thus there is some value $u \in V$ that appears $|T_j| - f$ times in T_j. By Lemma 11, u appears $|T^C| - f$ times in T^C. By the common core property of gather, u also appears $|T^C| - f$ times in T_i. Since $|T^C| - f = n - 2f$ and $n > 3f$, u appears $f + 1$ times in T_i. By Lemma 16, u is the only non-$\perp$ value that appears $f + 1$ times in T_i. Thus p_i sets v_i to u in Line 5. Since $u \neq \perp$ and u is v, $v \neq \perp$. □

Theorem 18. *Algorithm 4 is correct for any value of $R \geq 2$.*

Proof. *Termination* follows from the assumed correctness, including termination, of the G and AA subroutines.

Validity: Suppose correct process p_i decides (v, g).

(i) If $v \neq \perp$, we must show that some correct process has input v. By the code, v is some value that appears $f + 1$ times in T_i (cf. Line 5). At least one of these appearances of v is for a correct process p_j. By the validity property of gather, p_j makes its second call to G with input v, implying that v appears $|S_j| - f$ times in S_j (cf. Line 2). By Proposition 13, v is the input of a correct process.

(ii) If all correct processes have the same input, we must show $g = R$. Suppose all correct inputs have input v. Then they call G the first time with input v. By the validity property of gather, v appears $|S_j| - f$ times in S_j for each correct process p_j. It follows that each correct process calls G the second time with input v. Again by the validity property of gather, v appears $|T_j| - f$ times in T_j for each correct process p_j. Then each p_j sets v_j to v and r_j to R in Lines 5 and 6. By the validity property of approximate agreement, each correct process gets R as the output of AA and decides (v, R). Thus the g component in p_i's decision equals R.

Agreement: Suppose correct process p_i decides (u, g) and correct process p_j decides (v, h). By Lemma 17, $g = 0$ if and only if $u = \perp$, and $h = 0$ if and only if $v = \perp$.

Thus if g and h are both 0, we have $(u, g) = (v, h)$.

Suppose exactly one of g and h is 0, say g. By the 1-agreement property of approximate agreement, $h = 1$. Thus $(u, g) = (\perp, 0)$ and $(v, h) = (v, 1)$.

Suppose neither g nor h is 0. By the 1-agreement property of approximate agreement, $|g - h| \leq 1$. Since u appears $f + 1$ times in T_i and v appears $f + 1$ times in T_j (cf. Line 5), Lemma 16 implies $u = v$. □

5.2 Binding and its Preservation

Binding is a property of algorithms, first proposed for crusader agreement [4] and extended to connected consensus [10], which restricts the power of the adversary to influence later decisions once the first correct process decides. In more detail, the **binding property for connected consensus** states that for any execution prefix ending when the first correct process decides, there is a value $v \in V$ such that in every extension of α, no correct process decides (u, g) for any $u \neq v$.

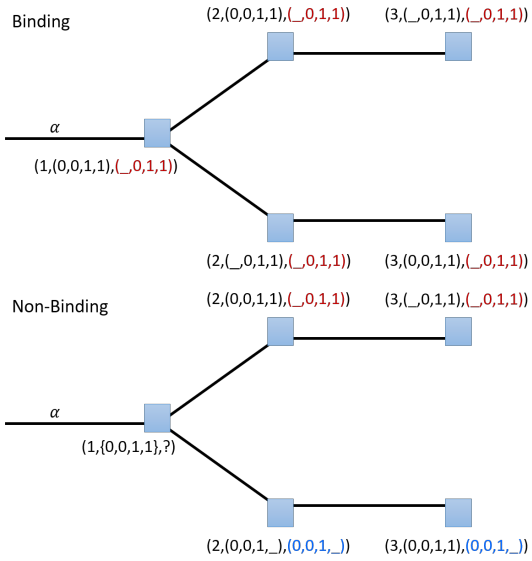


Figure 2: Binding vs. non-binding gather for $f = 1$, $n = 4$. The processes are $p_1, \ldots, p_4$ with p_4 being faulty. Black lines indicate executions with time increasing to the right. A square indicates a decision by a process and is labeled with the process' id, the process' output, and the common core ("?" if it is not yet fixed). The output and common core are denoted by a 4-vector with the i-th entry being the value for p_i, or underscore if missing. The common core consisting of the inputs of p_2, p_3, p_4 (resp., p_1, p_2, p_3) is colored red (resp. blue).

We define a similar property for gather, which restricts the power of the adversary to influence the common core once the first correct process finishes. In more detail, the **binding property for gather** states that for any execution prefix ending when the first correct process finishes, there is a set S^C of size $n - f$ such that in every extension of α the output of every correct process contains S^C.

To demonstrate the binding property for gather, consider Figure 2, in which α is an execution prefix that ends as soon as the first correct process, p_1, returns from gather with output $\langle 0, 0, 1, 1 \rangle$. The top of Figure 2 shows two different extensions of α when gather is binding. The common core is fixed to be $\langle _, 0, 1, 1 \rangle$ at the end of α, and in both the upper and lower extensions all the outputs contain the common core, even though they are not all identical. The bottom of Figure 2 shows two different extensions of α when gather is not binding. The common core is not fixed at the end of α. In the upper extension, the common core is $\langle _, 0, 1, 1 \rangle$ but in the lower extension, the common core is $\langle 0, 0, 1, _ \rangle$.

Figure 2 also elucidates the interaction between the binding property of gather and that of connected consensus in Algorithm 4 with $R = 1$ (crusader agreement). Recall that Algorithm 4 for $R = 1$ is

simply a call to a gather subroutine followed by some local computation. Suppose gather is binding and consider the top of Figure 2. The crusader agreement results in the upper extension are $\perp$ for p_1 and p_2, and 1 for p_3, while in the lower extension they are $\perp$ for p_1 and p_3, and 1 for p_2; these decisions satisfy the binding property for crusader agreement. Now suppose gather is not binding and consider the bottom of Figure 2. The crusader agreement results in the upper extension are $\perp$ for p_1 and p_2, and 1 for p_3, while in the lower extension they are $\perp$ for p_1 and p_3, and 0 for p_2. This violates the binding property for connected consensus since in the upper extension, 1 is decided while in the lower extension 0 is decided.

The next theorem shows that Algorithm 4 preserves the binding property of the gather subroutine used in it. A binding gather algorithm appears in [33] (see also [8]).

Theorem 19. *If the subroutine G satisfies binding for gather, then Algorithm 4 satisfies binding for connected consensus.*

Proof. We show an even stronger property for connected consensus: as soon as the first correct process returns from its call to G in Line 1, the branch for all decisions by correct processes in all extensions is fixed.

Let α be the prefix of any execution of the algorithm that ends as soon as the first correct process returns from its first call to G. By the binding property of gather, the common core S^C for the first instance of G is fixed at the end of α. By Proposition 12, there is at most one value in V that occurs $|S^C| - f$ times in S^C. Thus by Lemma 11 there is at most one value in V that occurs $|S_i| - f$ times in S_i, for any correct process p_i, in any extension of α.

Suppose $R = 1$. Since the branch decided on is either $\perp$ or the value that occurs $|S_i| - f$ times in S_i, the only possible non-$\perp$ branch decision in any extension of α is the value (if any) occurring $|S^C| - f$ times in S^C.

Suppose $R \geq 2$. Consider any extension α' of α in which a correct process p_i decides (v, g) where $v \in V$. By the code (cf. Line 5), v appears $f + 1$ times in T_i. At least one of these occurrences corresponds to a correct process p_j. By the validity property of gather, v is p_j's input to its second call to G on Line 4 (the value of variable v_j). By the code, v appears $|S_i| - f$ times in S_i. By Lemma 11, v appears $|S^C| - f$ times in S^C. As argued above for $R = 1$, there is only one possible choice for v and it is fixed at the end of α. □

Remark 1. *The core set S^C is hidden from the processes themselves. When gather is not binding, the core set is determined only in hindsight, and could be captured as a prophecy variable. When gather is binding, the core set is determined once the first gather returns, and thus, it becomes a history variable. (See [1] for a discussion of prophecy and history variables.)*

5.3 Comparison with Other Connected Consensus Algorithms

The result of instantiating Algorithm 4 with the binding gather subroutine [33] and one of the optimally-resilient approximate agreement subroutines in [2, 18] is an optimally-resilient connected consensus algorithm for arbitrary R that satisfies the binding property and has running time logarithmic in R. (The running time is analyzed in [8].) To the best of our knowledge, it is the first such algorithm, since the algorithm in [29] does not satisfy binding while the algorithm in [34] requires $n > 5f$ (and yet is not in canonical rounds).

Building blocks similar to connected consensus for arbitrary R have been proposed in earlier work on *synchronous* randomized algorithms for Byzantine agreement, for example k-proxcast in [19] and M-gradecast in [27]. These papers typically make various cryptographic assumptions such as the existence of broadcast channels or digital signatures.

6 Summary and Open Questions

Our results show that canonical-round algorithms, while useful for structuring and reasoning about asynchronous algorithms, fundamentally fail to capture the communication patterns required for optimal-resilience Byzantine fault tolerance. Even when randomization is allowed[1], canonical-round algorithms require an unbounded number of rounds in the regime $3f < n \leq 5f$, and communication-closed variants are unsolvable. This explains why known optimally-resilient Byzantine algorithms abandon strict round-based structure in favor of content-dependent primitives such as reliable broadcast and gather.

The gather primitive provides one concrete way to express the content-dependent communication patterns excluded by canonical rounds. We show that gather admits constant-time solutions with optimal resilience and supports simple reductions to tasks such as connected consensus, while preserving desirable properties such as binding.

Several open questions remain. First, the threshold $n = 5f$ marks a sharp boundary for bounded-round canonical-round algorithms in the Byzantine setting. It is natural to ask whether this threshold is strict: does every optimally-resilient algorithm admit a canonical-round formulation when $n > 5f$ as do randomized consensus [11], approximate agreement [22], and connected consensus [8]? Second, it would be interesting to understand whether similar limitations arise in other adversarial models, such as partially synchronous systems or settings with authentication, where equivocation can be avoided by other means.

Acknowledgments

Hagit Attiya and Itay Flam are supported by the Israel Science Foundation 22/1425 and 25/1849.

[1] Section 3 considers Las Vegas algorithms which have probabilistic termination. Essentially the same argument works for Monte Carlo algorithms which have probabilistic safety properties, as long as the probability of validity, ρ, is more than 1/2 and the probability of agreement is more than $2 - 2\rho$. This subsumes the case when both properties are satisfied with error at most 1/3. Related results for general algorithms and $n \leq 3f$ appear in [5].

References

[1] Martín Abadi and Leslie Lamport. 1991. The existence of refinement mappings. *Theoretical Computer Science* 82, 2 (1991), 253–284.

[2] Ittai Abraham, Yonatan Amit, and Danny Dolev. 2004. Optimal resilience asynchronous approximate agreement. In *Proceedings of the 8th International Conference on Principles of Distributed Systems (OPODIS)*. Springer-Verlag, Berlin, Heidelberg, 229–239. doi:10.1007/11516798_17

[3] Ittai Abraham, Naama Ben-David, Gilad Stern, and Sravya Yandamuri. 2024. On the Round Complexity of Asynchronous Crusader Agreement. In *27th International Conference on Principles of Distributed Systems (OPODIS)*, Vol. 286. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 29:1–29:21. doi:10.4230/LIPIcs.OPODIS.2023.29

[4] Ittai Abraham, Naama Ben-David, and Sravya Yandamuri. 2022. Efficient and Adaptively Secure Asynchronous Binary Agreement via Binding Crusader Agreement. In *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing (PODC)*. Association for Computing Machinery, New York, NY, USA, 381–391. doi:10.1145/3519270.3538426

[5] Ittai Abraham and Kartik Nayak. 2021. Crusader Agreement with Error $\leq 1/3$ is Impossible for $n \leq 3f$ if the Adversary can Simulate. https://tinyurl.com/48sx52up.

[6] Hagit Attiya, Armando Castañeda, Dhrubajyoti Ghosh, and Thomas Nowak. 2025. Equivalence and Separation between Heard-Of and Asynchronous Message-Passing Models. arXiv:2511.21859 [cs.DC] https://arxiv.org/abs/2511.21859

[7] Hagit Attiya, Itay Flam, and Jennifer L. Welch. 2025. Brief Announcement: Communication Patterns for Optimal Resilience. In *39th International Symposium on Distributed Computing (DISC) (LIPIcs)*, Dariusz R. Kowalski (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 46:1–46:7. doi:10.4230/LIPICS.DISC.2025.46

[8] Hagit Attiya, Itay Flam, and Jennifer L. Welch. 2026. Why Canonical Rounds Fail for Optimal Byzantine Resilience. arXiv:2510.04310 [cs.DC] https://arxiv.org/abs/2510.04310

[9] Hagit Attiya and Jennifer L. Welch. 2004. *Distributed computing - fundamentals, simulations, and advanced topics* (2nd ed.). Wiley, Hoboken, NJ.

[10] Hagit Attiya and Jennifer L. Welch. 2024. Multi-Valued Connected Consensus: A New Perspective on Crusader Agreement and Adopt-Commit. In *27th International Conference on Principles of Distributed Systems (OPODIS)*, Vol. 286. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 6:1–6:23. doi:10.4230/LIPIcs.OPODIS.2023.6

[11] Michael Ben-Or. 1983. Another Advantage of Free Choice: Completely Asynchronous Agreement Protocols. In *Proceedings of the Second Annual ACM Symposium on Principles of Distributed Computing*. Association for Computing Machinery, New York, NY, USA, 27–30. doi:10.1145/800221.806707

[12] Matthias Biely, Bernadette Charron-Bost, Andreas Gaillard, Martin Hutle, André Schiper, and Josef Widder. 2007. Tolerating Corrupted Communication. In *Proceedings of the 26th Annual ACM Symposium on Principles of Distributed Computing (PODC)*. Association for Computing Machinery, New York, NY, USA, 244–253. doi:10.1145/1281100.1281139

[13] Gabriel Bracha. 1987. Asynchronous Byzantine agreement protocols. *Information and Computation* 75, 2 (1987), 130–143. doi:10.1016/0890-5401(87)90054-X

[14] Christian Cachin, Klaus Kursawe, Frank Petzold, and Victor Shoup. 2001. Secure and Efficient Asynchronous Broadcast Protocols. In *Advances in Cryptology (CRYPTO)*. Springer-Verlag, Berlin, Heidelberg, 524–541.

[15] Ran Canetti and Tal Rabin. 1993. Fast asynchronous Byzantine agreement with optimal resilience. In *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing (STOC)*. Association for Computing Machinery, New York, NY, USA, 42–51.

[16] Bernadette Charron-Bost and André Schiper. 2009. The Heard-Of Model: Computing in Distributed Systems with Benign Faults. *Distributed Computing* 22, 1 (2009), 49–71. doi:10.1007/s00446-009-0084-6

[17] Benny Chor and Cynthia Dwork. 1989. Randomization in Byzantine Agreement. *Adv. Comput. Res.* 5 (1989), 443–497.

[18] B. A. Coan. 1988. A Compiler that Increases the Fault Tolerance of Asynchronous Protocols. *IEEE Trans. Comput.* 37, 12 (Dec. 1988), 1541–1553. doi:10.1109/12.9732

[19] Jeffrey Considine, Matthias Fitzi, Matthew K. Franklin, Leonid A. Levin, Ueli M. Maurer, and David Metcalf. 2005. Byzantine Agreement Given Partial Broadcast. *J. Cryptol.* 18, 3 (2005), 191–217. doi:10.1007/S00145-005-0308-X

[20] Andrei Damian, Cezara Drăgoi, Alexandru Militaru, and Josef Widder. 2019. Communication-closed asynchronous protocols. In *International Conference on Computer Aided Verification*. Springer-Verlag, Berlin, Heidelberg, 344–363.

[21] Danny Dolev. 1982. The Byzantine generals strike again. *Journal of Algorithms* 3, 1 (1982), 14–30.

[22] Danny Dolev, Nancy A. Lynch, Shlomit S. Pinter, Eugene W. Stark, and William E. Weihl. 1986. Reaching approximate agreement in the presence of faults. *J. ACM* 33, 3 (1986), 499–516.

[23] Cezara Drăgoi, Constantin Enea, Burcu Kulahcioglu Ozkan, Rupak Majumdar, and Filip Niksic. 2020. Testing consensus implementations using communication closure. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–29.

[24] Tzilla Elrad and Nissim Francez. 1982. Decomposition of distributed programs into communication-closed layers. *Science of Computer Programming* 2, 3 (1982), 155–173. doi:10.1016/0167-6423(83)90013-8

[25] Alan David Fekete. 1994. Asynchronous Approximate Agreement. *Information and Computation* 115, 1 (1994), 95–124.

[26] Pesech Feldman and Silvio Micali. 1997. An Optimal Probabilistic Protocol for Synchronous Byzantine Agreement. *SIAM J. Comput.* 26, 4 (1997), 873–933.

[27] Juan A. Garay, Jonathan Katz, Chiu-Yuen Koo, and Rafail Ostrovsky. 2007. Round Complexity of Authenticated Broadcast with a Dishonest Majority. In *48th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*. IEEE Computer Society, USA, 658–668. doi:10.1109/FOCS.2007.44

[28] Allison B. Lewko. 2011. The Contest Between Simplicity and Efficiency in Asynchronous Byzantine Agreement. *CoRR* abs/1106.5170 (2011), 21 pages. arXiv:1106.5170 http://arxiv.org/abs/1106.5170

[29] Mose Mizrahi Erbes and Roger Wattenhofer. 2025. Quit-Resistant Reliable Broadcast and Efficient Terminating Gather. In *28th International Conference on Principles of Distributed Systems (OPODIS)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 15:1–15:22. doi:10.4230/LIPIcs.OPODIS.2024.15

[30] Thomas Nowak and Joel Rybicki. 2019. Byzantine Approximate Agreement on Graphs. In *33rd International Symposium on Distributed Computing (DISC)*, Vol. 146. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 29:1–29:17. doi:10.4230/LIPIcs.DISC.2019.29

[31] Marshall C. Pease, Robert E. Shostak, and Leslie Lamport. 1980. Reaching Agreement in the Presence of Faults. *J. ACM* 27, 2 (1980), 228–234. doi:10.1145/322186.322188

[32] Gilad Stern and Ittai Abraham. 2021. Living with Asynchrony: the Gather protocol. https://tinyurl.com/4ujdk3p8.

[33] Gilad Stern and Ittai Abraham. 2024. Gather with Binding and Verifiability. https://tinyurl.com/462e52wc.

[34] Alan Ernesto Arteaga Vázquez. 2025. On Time-Optimal, Fault-Tolerant Algorithms for Connected Consensus Beyond Grade Two. In *29th International Conference on Principles of Distributed Systems (OPODIS)*, Vol. 361. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 24:1–24:28. doi:10.4230/LIPICS.OPODIS.2025.24